

Towards an Effective Connected and Automated Vehicle Capture The Flag Competition

Anonymous Author(s)

Abstract

The Connected and Autonomous Vehicles (CAV) ecosystem is rapidly transitioning from research testbeds to early real-world deployment. This evolution creates an urgent need for security training environments that capture the distributed, networked, and cyber-physical nature of modern CAV systems. However, existing automotive capture-the-flag (CTF) competitions largely focus on traditional in-vehicle components and often rely on proprietary platforms or specialized hardware, limiting their accessibility, reproducibility, and scalability.

To address this gap, we first perform the first systematic analysis of the CAV ecosystem to identify key threat models and attack surfaces, spanning in-vehicle components, V2X communication, and infrastructure-level interactions. Guided by this analysis, we design *ConCAVe*, a CARLA-based, containerized, and horizontally scalable CTF platform that supports challenges across diverse threat models. *ConCAVe* provides on-demand simulator instances and an extensible challenge authoring framework for building realistic and reproducible CAV security exercises.

We evaluate *ConCAVe* by conducting a CTF with 31 teams distributed across multiple nodes. The results indicate that *ConCAVe* is effective at providing a practical and accessible CAV security training environment: participants were able to set up and begin solving challenges within 30 minutes, and none reported the remote access model as a barrier. At the same time, our study surfaces key open problems (simulator instability) that highlight limitations in current infrastructure and point to directions for future research.

Overall, our findings identify critical infrastructure primitives necessary for scalable and accessible CAV security training and expose gaps that must be addressed to enable broader participation. To facilitate further research and community adoption, we open-source the *ConCAVe* platform and its associated challenge suite.

Keywords

connected and automated vehicles, V2X, capture the flag, cybersecurity education

ACM Reference Format:

Anonymous Author(s). 2026. Towards an Effective Connected and Automated Vehicle Capture The Flag Competition. In *Proceedings of Submission Draft*. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Modern Connected and Autonomous Vehicles (CAV) [19, 23] represent a fundamental shift in transportation. Unlike self-driving cars, which rely primarily on onboard perception to navigate, a CAV continuously exchanges messages with surrounding entities

such as traffic lights, roadside infrastructure, and other vehicles, and uses these messages to make driving decisions like adjusting speed, yielding, or halting at an intersection. CAVs are therefore substantially more automated than self-driving cars, since vehicle behavior depends not only on what the car senses but also on what the surrounding ecosystem reports. Cities worldwide are rapidly deploying CAV infrastructure, with Vehicle-to-Everything (V2X) communications enabling cooperative driving, intelligent traffic management, and real-time hazard awareness [47]. This connectivity, however, also expands the attack surface of the vehicle ecosystem beyond internal networks. In a CAV ecosystem, an attack may begin with the compromise of any participating element: an in-vehicle component, an onboard unit, a roadside unit, a backend service, or the communication channel itself. Once one element is under attacker control, the effects of that compromise can propagate through Vehicle-to-Everything (V2X) communication, causing other vehicles or infrastructure to act on malicious, delayed, or falsified information. Recent incidents [34, 14], including abuse of Waymo's systems [16], have demonstrated real-world attack vectors of automotive security failures, ranging from remote vehicle immobilization to infrastructure spoofing.

Despite the growing importance of CAV, our analysis of existing CTF platforms reveals an important gap: there are no hands-on training materials for understanding and practicing CAV security concepts at scale. Capture The Flag (CTF)s are an effective way to close this gap. Gamified training using CTFs has emerged as a successful approach for developing cybersecurity skills, with CTF competitions shown to improve both engagement and practical competence [33, 52, 32]. They provide structured difficulty progression, realistic attack scenarios, and competitive motivation that traditional coursework often lacks.

Our goal is to develop a scalable and effective CAV CTF platform that trains participants on the key attack surfaces of connected vehicle security.

Existing automotive CTFs fall short of this goal. As shown in Table 1, the majority of current competitions cover traditional in-vehicle attacks such as Controller Area Network (CAN) bus exploitation and Electronic Control Unit (ECU) firmware analysis. While these are important concepts they only cover a subset of the broader V2X threat model (Section 4.2). They do not include comprehensive V2X scenarios from Vehicle-to-Infrastructure (V2I), Vehicle-to-Vehicle (V2V), or Vehicle-to-Network (V2N) communication. Furthermore, many of them depend on physical hardware, limiting participation to small, in-person groups and making reproducibility difficult. Even the more accessible platform like VSEC[53] are still primarily automotive CTFs rather than open, software-based CAV CTFs. They do not jointly provide a reusable platform for modeling attacks that span vehicles, roadside infrastructure, backend services, and V2X communication across Vehicle-to-Infrastructure (V2I),

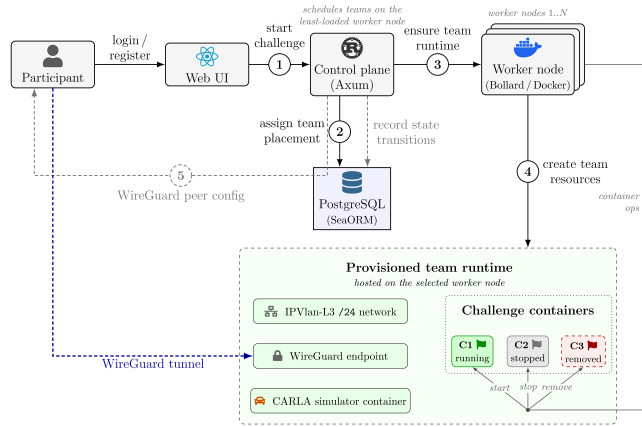


Figure 1: ConCAVe, our CTF platform for Connected and Autonomous Vehicle (CAV) security training. It shows the separate containerized environment for each team (which includes the vehicle simulator, CARLA), the communication channel for each team with its container, and the shared orchestration environment.

Vehicle-to-Vehicle (V2V), and Vehicle-to-Network (V2N) scenarios. This leaves a gap between existing car-hacking competitions and the hands-on training environments needed for realistic CAV security.

We argue that an effective CAV CTF must satisfy six requirements: good *progression* of challenges, *completeness* of threat coverage, *realism* of scenarios, *scalability* for participants without specialized hardware, *reproducible* challenges, and *sustain engagement* throughout the competition. Section 3.1 discusses each in detail. Neither fully hardware-bound competitions nor purely abstract software exercises satisfy all six requirements. What is needed is a middle ground that preserves the communication structure and domain-specific behavior of CAV systems while leveraging simulation for scalability.

To address this need, we present **ConCAVe**, a container-based platform for conducting CAV CTFs using isolated, on-demand digital twin environments. The architecture is depicted in Figure 1. Each participating team receives a dedicated CARLA vehicle simulator [11] and a simulated V2X communication layer, all accessible via a WireGuard [10] Virtual Private Network (VPN) tunnel and a web interface. The platform is organized around a control plane for orchestration, a challenge engine integrated with CARLA, and a web frontend for participant interaction. This design lets participants work in realistic connected-vehicle scenarios without requiring specialized hardware, while still preserving team isolation, reproducibility, and modular deployment.

We conducted the first iteration of a CAV CTF using **ConCAVe** with 31 teams. The challenge set covered attack vectors such as message interception, firmware exploitation, weak cryptography, and false-message injection. Our survey-based evaluation indicates that **ConCAVe** is an effective training tool. Participants reported that they prefer the on-demand nature of simulation and challenge environments and also had smooth experience with the network

infrastructure of the CTF. The evaluation also surfaced open problems, like lack of efficient and stable simulators for challenge development, that motivate future work. Our work sheds light on the design space of CAV CTFs and provides a concrete framework for building them, enabling future research on training and evaluation in connected-vehicle security.

In summary, we make the following contributions:

- We present a systematic threat model for CAV CTFs, identifying attack vectors across V2I, V2V, V2N, and in-vehicle communication surfaces.
- We survey existing automotive CTFs and identify key gaps in V2X coverage, scalability, realism, and reproducibility.
- We design and implement **ConCAVe**, a scalable, open-source CAV CTF platform with integrated V2X simulation.
- We evaluate **ConCAVe** through a live CTF with 31 teams, demonstrating reliable multi-node provisioning, low-friction remote access, and broad participant engagement, while surfacing simulator stability as a key open problem for CAV CTFs.

2 Background

Connected and Autonomous Vehicles (CAVs) [47] represent an emerging class of vehicular systems that combine automated driving capabilities with networked communication among vehicles, roadside infrastructure, and cloud services. In a typical CAV scenario, vehicles approaching an intersection communicate with one another to coordinate right-of-way decisions without relying on explicit traffic signals or stop signs. For example, vehicles can exchange messages to determine which one proceeds first, enabling efficient and autonomous intersection management. This contrasts with conventional autonomous vehicles, where driving decisions are locally determined. In such systems, a vehicle relies on onboard perception (e.g., image recognition) to detect a stop sign, halt, and then proceed after observing surrounding traffic, without direct coordination with other vehicles.

Tightly coordinated, fully automated driving requires vehicles to interact with roadside infrastructure (e.g., charging stations) via Vehicle-to-Infrastructure (V2I), with cloud-based services (e.g., monitoring or traffic management systems) via Vehicle-to-Network (V2N), and with other vehicles via Vehicle-to-Vehicle (V2V). Collectively, these communication paradigms are referred to as Vehicle-to-Everything (V2X). Figure 2 illustrates a typical CAV ecosystem. Where the On-Board Unit (OBU) is responsible for generating and transmitting V2X messages from the vehicle. Roadside Units (RSUs) are deployed at intersections and along roadways to broadcast infrastructure-related information such as Signal Phase and Timing (SPaT) data and road topology, allowing vehicles to adapt to their environment. At a higher level, the Operational Data Environment (ODE) aggregates this data to enable region-wide traffic management decisions. Underlying these interactions is the Security Credential Management System (SCMS), which manages digital certificates to ensure authentication and integrity of V2X communications. These messages are typically transmitted using Dedicated Short-Range Communication (DSRC) [55] or Cellular Vehicle-to-Everything (C-V2X) [54] technologies operating in the 5 GHz spectrum. Deployment of CAV infrastructure is accelerating.

The U.S. Department of Transportation’s ITS Joint Program Office has funded multiple connected vehicle pilot programs [48], and several states have invested in deploying V2X-enabled RSUs at signalized intersections [49]. Notably, the National Science Foundation has recently funded major initiatives [8] aimed at strengthening the security posture of the CAV ecosystem.

The high degree of automation and connectivity in CAV introduces significant new security risks. Vulnerabilities in one component can propagate across the ecosystem, potentially leading to large-scale and safety-critical consequences. Recent incidents [34, 14] highlight these risks, motivating increased attention from both academia and government.

2.1 Capture The Flags (CTFs)

CTF competitions [45] have emerged as a powerful pedagogical tool in cybersecurity education [37, 31], transforming abstract security concepts into engaging, hands-on learning experiences. Through gamified challenges, CTFs enable participants to develop practical skills by solving realistic security problems in a competitive setting. Today, CTF competitions are widely adopted by academia, industry, and the security community, with at least one major event occurring nearly every week [7].

CTFs generally follow three primary formats:

Jeopardy-style CTFs present a collection of independent challenges across diverse categories (*e.g.*, reverse engineering, cryptography, web exploitation), where teams earn points for each solved task. This format is the most common due to its simplicity and scalability, and is widely used in major competitions such as DEF CON CTF [43].

Attack-Defense CTFs assign each team an identical set of vulnerable services. Teams must simultaneously defend their own services by identifying and patching vulnerabilities, while actively exploiting weaknesses in other teams’ services. These competitions are highly dynamic and typically shorter in duration. A prominent example is the iCTF hosted by UC Santa Barbara [20].

King of The Hill (KoTH) CTFs focus on control over shared resources, where teams compete to maintain dominance for as long as possible. Scoring is based on both the duration of control and successful attacks against competing teams. r2wars [38] is an example of KoTH where programs compete to be the last one standing in a shared memory space.

3 Effective CAV CTF

Given the rapid advancement of the CAV ecosystem, it is critical to understand emerging threats and to develop appropriate pedagogical resources for training practitioners in CAV security. In particular, the distributed and highly interconnected nature of CAV systems introduces new attack surfaces that are not adequately captured by existing security training platforms.

This paper addresses the problem of designing an effective CAV CTF. We begin by identifying the key requirements (§ 3.1) and analyzing existing CTF frameworks (§ 3.2) to determine whether they meet them. Our analysis shows that current platforms are largely designed for **traditional or standalone automated vehicle** settings and do not adequately capture the broader security challenges inherent to the CAV ecosystem. To bridge this gap, we first

systematically identify relevant threat models and attack surfaces in CAV environments (§ 4.2). Guided by this analysis, we design *ConCAVe* (§ 4.3), a CAV CTF framework that satisfies the identified requirements and enables realistic, scalable, and extensible security training.

3.1 Requirements for an Effective CAV CTF

Several prior studies [30, 39, 5, 29] have explored the design principles required for CTFs to be effective learning tools. Most successful CTFs implicitly follow these guidelines. For instance, DEFCON CTF always has challenges with increasing difficulty.

We contextualize these principles for the CAV ecosystem and derive the following requirements for an effective CAV CTF:

- **Progression.** Challenges should follow a carefully designed difficulty curve. Given that CAV security is a relatively new domain, many participants may lack prior exposure to its concepts. A structured progression reduces participant dropout and supports gradual mastery of advanced topics [33].
- **Completeness.** The CTF should cover a broad spectrum of attack scenarios across multiple CAV threat models, while also enabling easy addition of new challenges. This is essential for long-term sustainability, as demonstrated by widely adopted CTF platforms [6, 37, 36].
- **Realism.** The CTF should reflect realistic CAV operating conditions. While full hardware testbeds are not required, participants must engage with representative scenarios, including V2X message flows, vehicle–infrastructure interactions, timing behaviors, and software components used in practice. Without such realism, participants risk learning isolated techniques without understanding how attacks manifest in real deployments. **Machiry: Cite relaven works**
- **Reproducibility.** The platform should be reproducible and easily accessible to both organizers and participants. Challenges must be rehostable and modifiable without reliance on proprietary infrastructure. Participants should be able to engage using commodity hardware and open-source software with minimal setup overhead, which is particularly important in educational settings [32].
- **Scalability.** The platform should support a large number of teams without degrading the participant experience. It must provide strong isolation guarantees to prevent interference between teams, a common issue observed in large-scale CTFs [52].
- **Sustained Engagement.** The CTF should maintain participant motivation through clear scoring mechanisms, real-time feedback, and seamless access. Prior operational studies show that usability and immediate feedback are critical factors in sustaining engagement throughout the competition [52].

3.2 Current State of the CAV CTFs

We conducted a comprehensive survey of existing automotive-security CTFs and related platforms to evaluate whether they satisfy the requirements outlined above and adequately support CAV threat models.

Table 1 summarizes the current landscape of automotive-security CTFs across dimensions most relevant to a CAV-focused training platform, including setup type, openness, fidelity, V2X coverage,

CTF Name	Setup Type	Source	Fidelity	V2X	Scale	CTF Style	Cost	Repro.	# Chals.
Rhme3 [40]	Hybrid	Open-Source	Medium	✗	Medium	Jeopardy	Medium	✓	19
Toyota’s Hack Festa [46]	Hybrid	Open-Source	High	✗	Medium	Jeopardy	High	✓	N/R
IVI CTF	In-Person	Open-Source	Medium	✗	Large	Jeopardy	Low	✓	N/R
CAN CTF	In-Person	Closed-Source	Medium	✗	Small	Jeopardy	High	✗	N/R
Defcon Automotive CTF [4]	In-Person	Closed-Source	High	✗	Medium	Jeopardy	High	Partial	N/R
VicOne x Blockharbor CTF [51]	In-Person	Closed-Source	High	✓	High	Jeopardy	High	Partial	40+
Pwn2own [57]	In-Person	Closed-Source	High	✗	High	Target-based	High	✗	76 [†]
ConCAVe (ours)	Online	Open-Source	Medium	✓	High	Jeopardy	Low	✓	7

Table 1: Comparison of automotive-security CTFs. ConCAVe is the only platform that is simultaneously open-source, remotely accessible, and V2X-capable. # Chals.: documented challenge-set size (N/R: not reported; [†]Pwn2Own is target-based, so we report the zero-days demonstrated at the 2026 event).

scalability, cost, and reproducibility. The comparison shows that, while existing efforts have made meaningful progress toward enabling practical automotive-security education, they remain only a partial fit for the broader CAV ecosystem.

Most existing automotive CTFs primarily target traditional in-vehicle attack surfaces. These platforms are valuable in lowering the barrier to hands-on training and enabling reuse beyond a single event. However, their scope is largely limited to internal vehicle networks. As a result, they do not capture the full CAV threat landscape, where attacks span interactions among vehicles, roadside units, backend services, wireless communication channels, and trust-management infrastructure. Where challenge-set sizes are publicly documented (Table 1), open competitions range from roughly 19 challenges in Rhme3 [40] to the 40+ of the closed-source VicOne/Block Harbor event [51]; even these larger sets concentrate almost entirely on in-vehicle surfaces rather than V2X.

At the other end of the spectrum, high-fidelity events such as Pwn2Own Automotive [18], the DEF CON Car Hacking Village [4], and manufacturer- or vendor-hosted CTFs provide access to realistic automotive systems. These events expose participants to real hardware and production-like environments, often enabling expert-level vulnerability discovery. However, they are typically expensive, conducted in-person, and difficult to reproduce. For example, Pwn2Own focuses on vulnerability discovery for expert researchers rather than serving as a reusable educational platform. Similarly, on-site vehicle CTFs hosted by automotive companies [9, 24] offer high realism but depend on specialized hardware and infrastructure, limiting scalability and reuse.

Commercial and industry-supported platforms, such as VicOne and Blockharbor, move closer to supporting V2X training by offering broader challenge sets and, in some cases, online accessibility. However, these platforms are typically closed-source or only partially reproducible. This restricts researchers and educators from inspecting challenge design, modifying environments, or rehosting the platform for independent use. Even when V2X-related content is

included, the lack of open and extensible infrastructure limits their effectiveness as a foundation for CAV CTF research and education.

Existing automotive-security CTFs provide only partial coverage of the CAV threat landscape, and there currently exists no open-source, reproducible platform for hosting CAV-focused CTFs.

4 Methodology

As established in Section 3.2, existing automotive CTF platforms provide useful training for selected attack surfaces, but they do not jointly satisfy the CAV-specific requirements defined in Section 3.1. To address this gap, we designed ConCAVe, a modular CAV CTF platform that combines isolated team networks, CARLA-based simulation, reusable V2X challenge primitives, and a web-based competition interface. Rather than adapting an existing CTF platform around these constraints, ConCAVe treats simulation, network isolation, remote access, and challenge orchestration as first-class platform responsibilities. We then deployed a preliminary IRB-approved CTF with beginner-level V2X scenarios to evaluate participant engagement, usability, and learning outcomes.

4.1 Design Challenges for CAV CTF

Satisfying all the requirements in Section 3.1 simultaneously is technically challenging. A CAV CTF platform must model vehicles and infrastructure while also operating as a conventional CTF system: teams need isolated, reproducible environments that can be provisioned, accessed, monitored, and reset during a live event.

A practical CAV CTF must therefore solve several problems simultaneously: running CARLA without a GPU-backed display, exposing V2X interactions without radios or roadside hardware, isolating teams from one another, scaling containers and networks

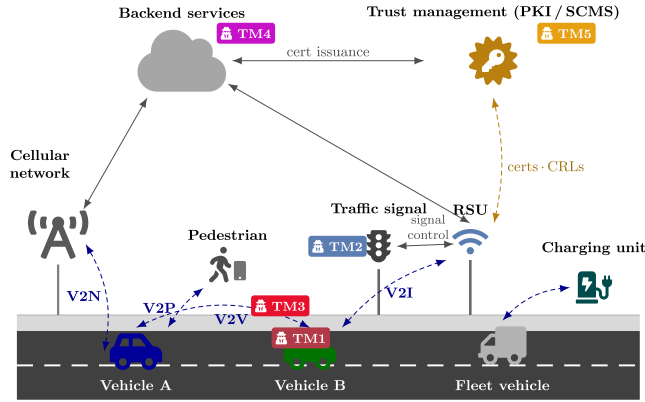


Figure 2: CAV Ecosystem and Threat Model 🚗 **vehicle-side compromise**, 🚦 **roadside-infrastructure compromise**, 🚗⚡ **wireless-channel manipulation**, ☁️ **backend-service compromise**, and ⚙️ **trust-management compromise**.

across machines, and giving participants access through commodity software rather than specialized equipment. We first define the adversary capabilities the platform must support, then describe the system design that realizes those capabilities in a live competition setting.

4.2 Threat Model

A CAV security training platform must represent attacks as they arise in the real operating context of connected vehicles: vehicles moving on roads, communicating with nearby vehicles, roadside infrastructure, traffic-control systems, backend services, and trust-management infrastructure. We therefore model the CAV ecosystem as a distributed cyber-physical system in which an attacker may control, observe, or manipulate one or more components that influence safety-relevant vehicle behavior.

Figure 2 illustrates this model. Each colored attacker symbol denotes a threat-model class, and placing that symbol on a component indicates that the attacker has control of, or influence over, that component under that threat model. A component may carry multiple attacker symbols when it is relevant to multiple threat models. For example, a traffic signal may be affected under a roadside-infrastructure compromise, a wireless-channel attack, or a trust-management attack.

We define five threat-model classes, each corresponding to a component class in the CAV ecosystem.

- **TM1: Vehicle-side compromise.** The attacker controls a vehicle-side component such as an OBU, sensor, autonomy module, ECU, or in-vehicle network. This model captures attacks in which a malicious or compromised vehicle can generate plausible V2X messages, manipulate local perception or planning state, or pivot from an externally reachable interface into safety-critical vehicle software.
- **TM2: Roadside-infrastructure compromise.** The attacker controls roadside infrastructure such as an RSU, traffic signal controller, intersection-management service, or roadside

charging unit. This model captures attacks that manipulate infrastructure-to-vehicle information, signal timing, local coordination messages, or infrastructure firmware in ways that affect nearby vehicles.

- **TM3: Wireless-channel manipulation.** The attacker controls the communication channel rather than a legitimate endpoint. This includes eavesdropping, replaying, injecting, delaying, or dropping messages on V2V, V2I, V2N, or Vehicle-to-Pedestrian (V2P) links within the attacker’s radio or network reach. This model captures attacks where the adversary exploits message exposure, weak authentication, missing freshness checks, or insecure transport assumptions.
- **TM4: Backend-service compromise.** The attacker controls or abuses backend services such as telemetry pipelines, fleet-management APIs, software-update infrastructure, operational data environments, or cloud coordination services. This model captures attacks that manipulate global state, exfiltrate sensitive vehicle telemetry, alter configuration, or use backend access to influence vehicles and infrastructure.
- **TM5: Trust-management compromise.** The attacker controls or subverts trust-management mechanisms such as certificates, signing keys, revocation state, or message-validation logic. This model captures attacks in which otherwise well-formed messages or software artifacts are incorrectly accepted as trustworthy.

These threat models are not mutually exclusive. A realistic attack may combine multiple forms of control: for example, an attacker may compromise a roadside unit while also manipulating wireless messages, or may abuse backend access to issue trusted configuration updates. The purpose of the model is therefore not to assign every attack to exactly one category, but to make explicit which components the attacker controls and which security property is violated.

For example, an attacker could abuse V2I signal preemption at a smart intersection by forging an emergency-vehicle priority request. If the roadside unit or traffic controller accepts the message without sender authentication or freshness checks, it may grant an unauthorized green phase and disrupt normal traffic flow. In our threat model, this maps to the V2I link between vehicles and traffic infrastructure, covering 🚗 vehicle-side compromise, 🚗⚡ wireless-channel manipulation, and ⚙️ trust-management failure.

We use this threat model as the coverage model for *ConCAVe*. Each challenge is an instantiation of three elements: the attacker-controlled component shown in Figure 2, the security property being violated, and the platform primitive required to reproduce the attack. This framing allows **each challenge to be described directly in terms of the CAV ecosystem rather than as an isolated software exploit.**

4.3 System Design

To provide the platform primitives required by the threat model while satisfying the goals of isolation and scalability, *ConCAVe* uses the distributed architecture shown in Figure 1. The system consists of three main components: a control plane for orchestration, a Python challenge engine integrated with CARLA, and a web frontend for participant interaction. The control plane manages

Table 2: Connected and Autonomous Vehicles CTF requirements (Section 3.1) and the ConCAVe mechanism that realizes each.

Requirement	Mechanism
Progression	Challenge suite ordered from environment setup through message interception, infrastructure manipulation, and firmware exploitation (Table 3); the shared Challenge interface lets authors add graded challenges.
Completeness	Threat models TM1–TM5 serve as an explicit coverage model, backed by reusable V2X sensors, Cooperative Awareness Message (CAM) generation, and capability metadata for new challenges.
Realism	CARLA vehicle physics, a real per-team network stack, and ETSI-style CAM/V2X protocol logic, while abstracting radios, RSU hardware, and production Public Key Infrastructure (PKI).
Reproducibility	Containerized platform and challenge suite; each challenge can run standalone via docker compose up against CARLA, and the suite will be released as open source.
Scalability	Primary/worker control plane with sticky placement and a per-worker capacity cap; deterministic per-team subnets and ports let nodes provision independently, so capacity grows by adding workers.
Sustained Engagement	Web frontend with participant-controlled container lifecycle, live scoreboard and flag submission, and WireGuard access that requires only commodity software.

user authentication, container lifecycle, and network provisioning through the Docker API. For scalability, it supports multi-node deployments across remote compute nodes with health monitoring and automatic reconnection.

4.3.1 Design Goals. **Aditya: Setup the rest of the sections by stating the practicality-fidelity position and make a table out of 3.1 that we use to evaluate ConCAVe4**

ConCAVe deliberately occupies a middle ground between practical deployment and system fidelity. Physical testbeds such as PASTA [35] and RAMN [46] achieve high fidelity but bind participation to hardware and cannot scale to a CTF-sized event. Purely abstract software challenges scale more easily, but they discard the domain-specific behavior that makes CAV attacks distinctive. ConCAVe preserves the fidelity most relevant to CAV security training by retaining simulated vehicle physics, a real per-team network stack, and protocol-level V2X logic, while deliberately abstracting radios, RSU hardware, and production PKI. Every subsystem described below realizes one of the requirements established in Section 3.1. Table 2 makes this mapping explicit and serves as a guide to the rest of this section.

4.3.2 Architecture Overview. **Aditya: Concise data-flow description that touches all components**

Aditya: Concise data-flow description that touches all components

Figure 1 shows how the three components interact during a session. A participant registers and logs in through the web frontend, which communicates with the control-plane API on the primary node. When the participant starts a challenge, the primary

resolves the team’s placement, provisions the runtime on the assigned worker if it does not already exist, and returns a WireGuard [10] configuration. We chose WireGuard over OpenVPN or SSH-based tunneling because it is part of the mainline Linux kernel, has first-party clients on all major desktop operating systems, and reduces participant setup to importing a single configuration file. The worker hosts the team’s containers on a private subnet: a CARLA simulator, the challenge container, and a WireGuard endpoint. The participant activates the tunnel, connects the CARLA Python client to render the 2D world (Figure 3), and drives the simulated vehicle. The challenge’s V2X traffic reaches the participant’s tools through a WebSocket bridge, and messages the participant crafts are injected back into the simulation. Flag submissions return to the control plane over the same API, which records them in PostgreSQL and updates the scoreboard.

4.3.3 Control Plane and Runtime State. **Aditya: Expand with the provisioning state-machine and add db persistence model description**

The control plane is the authority for user state, challenge state, and runtime placement. It manages registration, login, challenge metadata, flag submission, scoreboard updates, container lifecycle operations, health checks, and administrative recovery actions. Persistent state is stored in PostgreSQL across a small set of tables: user accounts and session tokens, flag submissions and scores, registered nodes and their last-seen health, team-to-node placement, and a provisioning-job log. This makes team runtime state explicit rather than implicit in the Docker daemon on a single host.

When a participant starts a challenge runtime, the control plane resolves the team’s placement, provisions the required runtime if it does not already exist, and routes subsequent operations to the node that owns that team. Each team therefore has a stable runtime identity throughout the CTF. This stable identity matters during live events because it lets participants recover from local challenge failures without requiring organizers to manually inspect or recreate their environment.

Placement follows an explicit state machine recorded in the placement table. The first time a team starts a challenge, the scheduler assigns it to a worker and the placement enters assigned; setting up the network and containers moves it to provisioning, and a successful setup to ready. If the owning worker stops reporting health (Section 4.3.8), the placement is marked degraded, and administrative teardown moves it to destroyed. First-start provisioning runs synchronously under a per-team lock so concurrent starts cannot race over the same resources. The asynchronous job queue is reserved for administrative recovery, namely destroy, rebuild, and reset, with each step appended to the job log so operators have a timeline to inspect when a recovery action fails.

4.3.4 *Realistic CAV Environment.* The simulator provides the cyber-physical substrate for ConCAVe: it manages vehicles, roadside entities, vehicle dynamics, and the world state that challenges manipulate. We selected CARLA [11] as the simulation backbone because it is open source, exposes a comprehensive Python API, supports custom sensors, and can run as a server to which participant tools connect. We also considered LGSVL [41] and MetaDrive [28]; however, LGSVL is no longer actively supported, and MetaDrive was

less suitable for hosting challenge instances through a clean client-server architecture. CARLA has operational limitations, which we evaluate in Section 5.

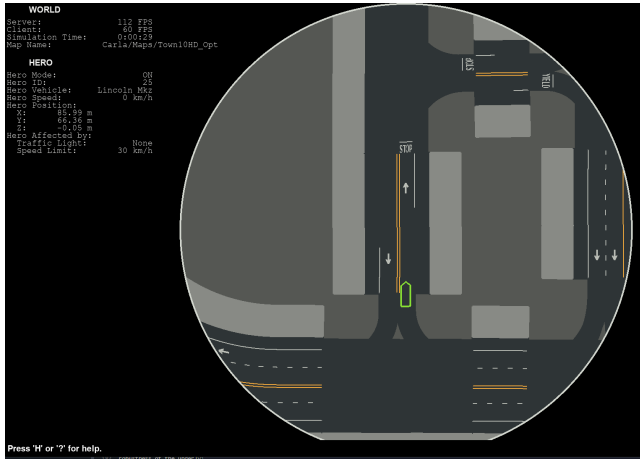


Figure 3: CARLA Client used to interface with the simulator.

We run CARLA in headless mode: the server advances the simulation and vehicle physics without rendering game entities. This allows a cloud compute node to host multiple simulator instances without GPU-backed displays. Participants connect to the simulator through CARLA’s Python API and render a lightweight 2D view of the game world, including nearby V2X entities, as shown in Figure 3. The UI represents the vehicle as a green polygon and displays position information alongside the map, giving participants enough situational awareness to drive through the scenario without requiring a full 3D client. This choice keeps setup cost low for beginners and lets them spend more time solving the security task rather than configuring simulation infrastructure.

4.3.5 V2X Communication Layer. **Aditya: Expand with detailed description of the python library, message transmission specifics, capability frames, websocket bridge.**

ConCAVe implements a lightweight V2X layer on top of CARLA’s Python API. Vehicles and roadside entities carry V2X sensors that broadcast CAM-style messages. Each message carries the sender’s identifier and station type, its position, heading, speed, acceleration, and yaw rate, and an extensions field that challenge authors use for challenge-specific data. A sensor delivers a message only to other sensors within a fixed range, and it emits messages following ETSI-style triggering rules [13]. This reproduces the burst-on-change, heartbeat-when-idle traffic pattern of real CAM dissemination, so that timing- and frequency-dependent behavior is visible to participants rather than abstracted away. Authors attach filters and handlers to a sensor to drop or react to received messages, and they implement the challenge’s completion check over the resulting V2X state.

Each sender also advertises a *capability frame* describing what it broadcasts and what it accepts. Every extension field is listed with its name, type, and whether it is required, together with the station type and broadcast range. Capability frames are the discovery mechanism of the platform, rather than guessing a challenge’s

wire format, a participant reads the advertised schema and knows exactly which fields a target will parse and validate.

A WebSocket bridge connects participant tools to the simulated network. On the inbound path it serializes observed CAM and capability frames to JSON and streams them to the player; on the outbound path it deserializes player-supplied CAM messages and injects them into the simulation through a virtual sensor bound to the player’s vehicle, so that crafted traffic is subject to the same range and delivery rules as legitimate traffic. Because the bridge speaks JSON over a standard WebSocket, participants can drive it from any language or from an interactive console, with no platform-specific client.

To further lower the entry barrier, ConCAVe ships a small Python client library, *vlib*, that wraps the bridge protocol. *vlib* exposes primitives to connect to a team’s bridge, enumerate visible senders and their capability frames, subscribe to a live feed of decoded CAM messages, and construct and inject well-formed messages with challenge-specific extensions populated. This lets participants focus on the security task (what to send and why) instead of on serialization, socket management, or the CAM field layout. *vlib* is the participant-facing tooling whose usability we evaluate in Section 5 (RQ3).

4.3.6 Team Isolation and Remote Access. **Aditya: Specificity slop**

Team isolation is enforced at the network layer. Each team is mapped to a deterministic /24 subnet derived from its team identifier, of the form $172.x.y.0/24$, so any node can compute a team’s addresses without a central registry or coordination between nodes. The layout within the subnet is fixed: the gateway at .1, the WireGuard endpoint at .5, the CARLA simulator at .11, and challenge containers in the .20-.253 range. Each team gets its own Docker network created with the IPVlan driver in L3 mode over a shared dummy parent interface. We chose IPVlan L3 over bridge networks or MacVLAN because it routes between subnets without a bridge per network or a unique MAC per container, and because it does not require publishing a host port for every challenge service.

Remote access is provided through a per-team WireGuard container. The web interface distributes the client configuration to the corresponding team, with its AllowedIPs restricted to that team’s subnet, so the tunnel reaches only that team’s services. The container publishes a single UDP host port, derived deterministically as $51800 + (\text{team_id} \bmod 10000)$. Routing all challenge services through one VPN port per team, rather than exposing each service on the host, keeps host port usage constant as teams are added and prevents one team from reaching another’s subnet.

4.3.7 Accessibility. ConCAVe’s web frontend gives participants controlled lifecycle operations for their own runtime. They can start, stop, and rebuild challenge containers on demand, which reduces organizer intervention when a container crashes or a team needs to inspect runtime state. These controls keep common recovery actions inside the participant workflow instead of turning every runtime failure into an operator support request.

4.3.8 Scalable Architecture. To scale beyond a single host, ConCAVe’s control plane runs in two modes: primary and worker. The primary node serves the UI and API and stores shared state in PostgreSQL, while workers host team runtimes. Nodes register with the

control plane, report a heartbeat every 15 seconds, and advertise runtime capabilities such as Docker, networking, and WireGuard support. When a team starts a runtime for the first time, the primary assigns it to the least-loaded healthy worker that is still below its team cap and records that placement; the assignment is sticky, so later container operations, logs, network-status requests, and WireGuard configuration requests all route to the same worker. Provisioning a runtime is idempotent: each step ensures the network and the WireGuard endpoint exist, so a re-run on a team that is already ready skips the resources that are present instead of recreating them.

The platform also treats failure as part of the runtime model. A reconciliation task on the primary marks a worker unhealthy once it has missed heartbeats for longer than a fixed timeout, roughly twice the heartbeat interval, and cascades the failure by marking that worker’s team placements as degraded. If provisioning stalls, the placement is similarly moved into a degraded state instead of appearing silently ready. Operators can then drain nodes, reset team resources, rebuild runtimes, or destroy stale resources through the administrative interface. These recovery paths are essential for CAV CTFs because simulator workloads are CPU-intensive and long-running, making partial runtime failures likely during live events.

The preliminary CTF ran on two AWS *c5.12xlarge* instances (48 vCPUs, 96 GiB RAM) at \$2.04/hour. CARLA’s CPU-intensive physics simulation makes the platform CPU-bound, so vertical scaling alone is inefficient; the platform instead scales horizontally by adding worker nodes. In our deployment, each compute node supported 16 teams, while a smaller instance ran the control plane and telemetry. Additional compute nodes can be added with teams deterministically assigned across the available infrastructure.

4.4 Challenge Architecture

Challenges are implemented using the provided Python framework, which wraps CARLA’s Python API with CTF-specific abstractions. Each challenge extends a common Challenge interface with lifecycle methods for setup, start, completion checking, and cleanup. The framework provides reusable V2X sensors, CAM-style message generation, capability metadata, message filters, and a WebSocket bridge. This lets challenge authors define attacker/victim roles, traffic infrastructure, message extensions, and completion logic without rewriting networking or simulator boilerplate.

The challenges are designed for reproducibility after the live CTF. Each challenge can operate independently of the competition infrastructure and requires only the CARLA simulator container. The platform and challenge suite are containerized and will be released as open source, allowing others to run the challenges locally with a single `docker compose up` command.

This decoupling is intentional: researchers and educators can use individual challenges without deploying the full *ConCAVe* infrastructure, and authors can develop and test new challenges in isolation before integrating them into a competition deployment.

To evaluate the platform’s capabilities, we developed a preliminary suite of reference challenges. Table 3 maps these challenges to the CAV threat model using four dimensions: attacker position, target component, adversarial goal, and required platform primitive.

This mapping emphasizes expressiveness rather than exhaustiveness. The preliminary challenge suite does not claim to cover all V2X attacks; instead, it demonstrates that the platform can instantiate multiple classes of CAV threats using common infrastructure primitives. Attacks requiring production PKI, standards-complete DSRC/C-V2X stacks, timing-accurate radio behavior, or full misbehavior detection remain outside the current implementation and are treated as future extensions rather than present capabilities.

5 Evaluation

Our evaluation seeks to answer the following research questions.

- **RQ1 (Completeness):** How complete is *ConCAVe* w.r.t. accommodating the entire threat model in V2X?
- **RQ2 (Extensibility):** How extensible is *ConCAVe* to add new challenges?
- **RQ3 (Usability):** What is the effort for both developers and participants to use *ConCAVe*?
- **RQ4 (Stability):** How stable is *ConCAVe* in organizing a real-world CTF?
- **RQ5 (Effectiveness):** How effective is *ConCAVe* for training participants on security aspects of V2x?

5.1 Survey

To evaluate the background of the participants and the learning outcomes, we conducted pre- and post-ctf surveys. Table 4 summarizes the main question categories included in the pre- and post-CTF surveys. The pre-ctf survey included baseline knowledge questions covering various V2X scenarios in which a vehicle could be in. The post-ctf survey repeated the same scenarios again to see if the player perception had changed after working on the challenges. In addition, the post-ctf survey also collected feedback on the CTF-platform stability and usability.

Table 5 summarizes participant demographics. Most of them were Freshmen/Sophomore students. Participants had varied prior exposure to CTFs, but limited exposure to automotive and CAV-specific CTFs. In the pre-CTF survey, 58.1% of respondents reported having never participated in any CTFs, while only 1 person reported prior participation in automotive CTFs.

5.2 RQ1: How complete is *ConCAVe* w.r.t accommodating different threats in V2X?

5.2.1 Method. We use the five defined V2X threat-model classes 4.2, and check each threat model against the platform primitives: CARLA support, V2X message observation/injection, per-team isolation, attacker/victim roles, cyber-physical effects, timing/range constraints, scoring hooks, and repeatable deployment.

- **Native:** supported with existing platform primitives and challenge logic only.
- **Minor extension:** requires small additions to challenge code or libraries, but no control-plane or runtime changes.
- **Major extension:** requires new runtime, deployment, or infrastructure support.
- **Not supported:** outside the current platform assumptions.

The primary metric is the fraction of threat models rated as *Native* or *Minor extension*.

ID	Challenge	Threat Model Class	Attack Vector	Skills Tested
01	System Diagnostics	N/A	Setup Testing	Establish the CTF environment and use the provided tools
02	Echoes in the Wires	TM1, TM3	V2X Message Interception	Intercept V2X messages and communicate with infrastructure using provided libraries
03	Intern's Traffic Light	TM2, TM3	Traffic Signal Manipulation	Analyze traffic-light broadcasts and recover weakly protected V2I information.
04	Coordinated Chaos	TM1, TM3	Vehicle Coordination Attack	Control coordination between vehicles for platooning in simulation
05	Plug & Pwn	TM2, TM3	Firmware Exploitation	Reverse engineer charging station firmware and exploit vulnerabilities to propagate malicious code to vehicles
06	Speed Camera	TM1, TM3	GPS Spoofing	Send malicious packets to gain traffic signal priority
07	Botnet	TM1, TM3	DDoS	Infect multiple vehicles and use them to overload a roadside unit.

Table 3: CTF Challenges

Table 4: Summary of survey

Category	Description	# Questions
Pre-CTF		
Education qualification	Level of education and participant role	2
CTF experience	CTF playing and organization experience	9
CAV knowledge tests	Baseline knowledge of CAV/V2X security scenarios	11
Post-CTF		
Platform usability	Setup time, Infrastructure stability, documentation adequacy	6
CAV knowledge tests	Repeated baseline knowledge of CAV/V2X security scenarios	11
Suggestions	Suggestions for Improvements	3

Table 5: Participant background from the pre-CTF survey

Category	Count	Percentage
Student	31	100%
Freshmen/Sophomore	15	48.4%
Junior/Senior	5	16.1%
Master's student	6	19.4%
PhD student	5	16.1%

5.2.2 Results. Table 6 shows the completeness results for *ConCAVe* across the defined V2X threat models. Beyond marking each threat model as supported, the table grounds every rating in concrete *evidence*: for the three natively supported classes we list the implemented challenges that actually exercise them, so completeness is demonstrated by working challenges rather than asserted. Each of TM1, TM2, and TM3 is exercised by four distinct challenges

built entirely on existing platform primitives (CARLA actors, CAM exchange, the WebSocket bridge, per-team isolation, and completion checking), with no control-plane or runtime modifications. 3/5 defined threat-model classes are therefore natively supported.

TM4 (Backend Services) requires only a minor extension: the challenge author adds the backend service (fleet management, OTA, etc.) as a per-challenge container or API, reusing the same isolation and scoring hooks; no control-plane or runtime change is needed. We classify it as *Minor* rather than *Native* only because no backend challenge shipped in the preliminary suite, not because the platform lacks a mechanism for it. TM5 (Trust Management) is the one class requiring a major extension. *ConCAVe* does not implement a PKI or cryptographic primitives in the communication channel, which are required to model certificate management and signature verification in a V2X PKI. Challenge code can still model simplified, permissive trust checks, but full certificate-backed trust is out of scope for the current communication layer.

This coverage also sets *ConCAVe* apart from existing platforms. As Table 1 shows, the surveyed automotive CTFs are almost entirely confined to TM1-style in-vehicle attacks; only the closed-source VicOne/Block Harbor offering reaches any V2X content. *ConCAVe* natively spans TM1–TM3 in an open, reproducible form.

ConCAVe natively supports most V2X threat models (3/5), with lightweight extensions needed for backend services and more substantial additions required to support trust management.

5.3 RQ2: How extensible is *ConCAVe* to add new challenges?

5.3.1 Method. We evaluate extensibility by measuring the development effort needed to add each challenge to the base platform.

Table 6: Threat-model completeness. Status summarizes the level of platform support (*Native*: existing primitives suffice; *Minor*: challenge-code additions only; *Major*: new runtime/infrastructure). Evidence lists the implemented challenges that exercise each threat model in the preliminary CTF.

Threat model	Platform capability	Status	Evidence (challenges)	Limitation
TM1: Vehicle-Side Compromise	Compromised or malicious vehicle entities can observe, generate, and inject V2X messages	Native	Ch. 02 (<i>Echoes in the Wires</i>), 04 (<i>Coordinated Chaos</i>), 05 (<i>Plug & Pwn</i>), 07 (<i>Botnet</i>)	Custom bridges must be written for each new protocol.
TM2: Roadside Infrastructure Compromise	Entities such as traffic lights, RSUs, and charging infrastructure can be modeled as challenge-controlled services.	Native	Ch. 03 (<i>Intern’s Traffic Light</i>), 05 (<i>Plug & Pwn</i>), 06 (<i>Speed Camera</i>), 07 (<i>Botnet</i>)	None
TM3: Wireless Channel Manipulation	WebSocket V2X layer lets participants intercept, replay, inject, and manipulate messages between simulated entities.	Native	Ch. 02, 03, 05, 06	Custom message handlers must be defined.
TM4: Backend Service Compromise	ODE services can be added as per-challenge containers or APIs connected to the simulated vehicle environment for fleet-management-style challenges.	Minor	— (expressible via challenge-container APIs)	Backend logic must be implemented as part of the challenge code.
TM5: Trust Management Compromise	Challenge code can model simplified V2X trust checks, such as permissive message validation, without implementing a full certificate-backed trust system.	Major	— (requires PKI/SCMS substrate)	<i>ConCAVe</i> does not implement any PKI or SCMS in V2X channels.

Table 7: Challenge-development effort.

Challenge ID	LoC	Runtime modification
01	109	None
02	324	None
03	761	None
04	1,108	None
05	1,360	Minor
06	441	None
07	434	None

Specifically, we record the lines of challenge-specific implementation code and whether the challenge required runtime support beyond the existing CARLA, CAM, WebSocket, team isolation, and completion-checking primitives. This includes any setup scripts and build files as well.

5.3.2 Results. Across the implemented challenges, we reused the same platform-facing primitives: CARLA actors, CAM exchange, the WebSocket bridge, team isolation, and capability metadata. As Table 7 shows, most challenges required only scenario-level challenge logic and no runtime modification. Plug & Pwn (Challenge 05) was the only exception as it added firmware artifacts and a small native build/runtime task,

but did not require a new communication model or changes to the core control plane.

ConCAVe demonstrates strong extensibility, as most challenges require only scenario-level logic with no modifications to the core platform.

5.4 RQ3: What is the effort for both developers and participants to use *ConCAVe*?

5.4.1 Method. Reaching a challenge requires a participant to complete a chain of steps, any of which can become a barrier, so we evaluate usability over the whole path rather than a single step. Before attempting the security task a participant must:

- (1) register and log in through the web frontend;
- (2) start a challenge, which triggers control-plane provisioning of the team’s CARLA simulator, challenge container, and WireGuard endpoint (Section 4.3);
- (3) import a single team-scoped WireGuard configuration and bring up the tunnel;
- (4) attach the CARLA Python client to render the 2D world (Figure 3); and
- (5) use the `vlib` library and WebSocket bridge to observe and inject V2X messages.

No step requires specialized hardware, a local CARLA build, a 3D GPU client, or manual network configuration. We measure this path through the post-CTF survey with setup time to first solve attempt,

whether WireGuard hindered work, the vlib rating, and documentation adequacy. Because steps 2–4 are platform-automated, a low setup time reflects the combined onboarding, provisioning, VPN, and simulator-attach path rather than any single convenient step. Separately, we use control-plane telemetry to quantify organizer burden where lower counts indicate participants self-recovered through the frontend’s lifecycle controls instead of escalating to organizers.

5.4.2 Results. Table 8 shows that the end-to-end platform was not a major source of participant friction. Every post-survey respondent reached a first solve attempt within 30 minutes. This single figure covers the full path i.e., a fast first attempt implies the participant finished onboarding, had their runtime provisioned, brought up the WireGuard tunnel, and attached the CARLA client within the window. Consistent with this, no respondent reported that WireGuard hindered solving and zero infrastructure issues were reported, even though most participants had not previously used a VPN-tunneled remote simulator.

All the friction was concentrated in the challenge itself rather than in reaching it. Participants rated the vlib library moderately (median 6/10) and were divided on documentation adequacy (4/7 sufficient). The platform thus removed infrastructure and network-access effort but left room to improve the tooling used inside a challenge which we believe is easy to add since it needs better libraries and documentation rather than architectural change.

On the organizer side, deployment controls kept support work minimal for a 31-team event: only 0.21 rebuilds per team–challenge pair and two manual interventions across the entire competition. When a team did need attention, sticky placement let the organizer rebuild or migrate it to another node without disturbing other teams. We therefore read the deployment as successful at reducing the setup, network-access, and simulator-attach effort that has historically made CAV training hardware-bound and organizer-intensive, while showing that documentation depth and challenge-tool ergonomics remain the principal usability concerns.

Table 8: Usability metrics for ConCAVe

Metric	Source	Result
<i>Participant-facing</i>		
Setup time to first solve attempt (min)	Post-survey	< 30: 100%
Documentation adequacy	Post-survey	4/7 Yes; 3/7 No
Interaction library rating (1–10)	Post-survey	Median 6
Infrastructure Issues	Post-survey	0%
<i>Organizer-facing</i>		
Admin interventions	Telemetry	2
Avg. rebuilds per team–challenge pair	Telemetry	0.21

ConCAVe achieves low setup and access friction for participants, while highlighting documentation quality and challenge tooling as the primary remaining usability bottlenecks.

5.5 RQ4: How stable is ConCAVe in organizing a real-world CTF?

5.5.1 Method. We evaluate stability along two dimensions: the stability of the CTF infrastructure and the stability of the simulation environment exposed to participants. For infrastructure stability, we use control-plane telemetry to measure provisioning success, recovery success, provisioning latency, and node health during the event. These metrics capture whether organizers could create, reset, and maintain team environments without frequent manual intervention. For simulation stability, we use the post-CTF survey rating of simulator stability, which captures the participant-facing reliability of the CARLA-based challenge environment.

5.5.2 Results. Table 9 shows that the core *ConCAVe* infrastructure remained stable during the CTF. Container provisioning succeeded in **99.57%** of operations, recovery through team reset succeeded in 100% of observed cases, and the platform reported no unhealthy node events i.e., the controlplane had no issues scheduling team containers on the container nodes across the two-worker deployment, confirming that capacity scales horizontally by adding more workers (Section 4.3.8). Provisioning was also fast enough for interactive CTF operation, with a median latency of **1.04** seconds and a p95 latency of **10.38** seconds. The high p95 was exclusively due to CARLA container which is expected considering the heavy nature of the simulator. These results indicate that the control plane, node management, and reset mechanisms were reliable enough to support a live multi-team event.

The main stability issue was instead exposed at the simulator layer. Participants reported a median simulator stability score of **3/10**, indicating that CARLA-related failures or degraded simulator behavior were visible during challenge work even when the surrounding CTF infrastructure remained healthy. We therefore interpret RQ4 as a mixed result. *ConCAVe* demonstrates that a containerized, multi-node CTF control plane can support a live CAV security event, but the participant experience remains constrained by the operational robustness of the underlying simulator.

The control-plane infrastructure held up reliably and at scale for a live CAV CTF, but overall stability was limited by simulator-level issues.

5.6 RQ5: How effective is ConCAVe for training participants on security aspects of V2X?

5.6.1 Method. We evaluate effectiveness by comparing pre-CTF and post-CTF survey responses and by analyzing challenge performance. The pre-survey captures prior CTF experience, automotive CTF experience, CAV/V2X familiarity, and vehicle-network technologies known before the event. The post-survey captures

Table 9: Runtime stability metrics.

Metric	Source	Result
Provisioning success rate	Container operations	99.57%
Recovery success rate	Team reset	100%
Provisioning latency	Runtime operation timestamps	p50(1.04s) p95(10.38s)
Node health	Heartbeats	0 errors
Simulator stability	Post-survey (0–10) rating	Median 3

technologies participants report familiarity with after the event and perceived interest in the challenges.

We distinguish between three levels of participation. Registered teams are teams that signed up for the event. Active teams are teams that initiated at least one platform action during the event. Scoreboard teams are active teams that completed at least one challenge and therefore appeared on the scoreboard. This distinction lets us separate event attendance from challenge engagement and demonstrated challenge success.

5.6.2 Results. Table 10 separates registration, participation, and demonstrated challenge success. Of the 31 registered teams, 21 (67.7%) became active by initiating at least one platform action, and 16 of those 21 active teams (76.2%) reached the scoreboard by solving at least one challenge, with 5 active teams solving every challenge in the suite. These numbers form our primary effectiveness signal. A majority of the participants who engaged with *ConCAVe* went on to demonstrate a concrete V2X exploitation skill, even though most had no prior automotive- or CAV-CTF experience (Table 5).

We additionally compare pre- and post-CTF knowledge responses. Post-CTF respondents showed a small positive change (+6%) in baseline V2X security knowledge. We report this change as a preliminary and underpowered signal rather than a measured learning gain. Only 6 of the 21 active teams completed the post-CTF knowledge survey, so the sample is too small to support a statistical claim. Increasing post-event response rates is a priority for strengthening this measurement in future iterations.

Most active participants moved from little or no prior CAV-CTF experience to a demonstrated V2X exploit. Self-reported knowledge gains are directionally positive but underpowered ($n=6$) and reported as preliminary.

6 Related Work

Existing research on CAVs falls into two categories: automotive security competitions and testbeds, and V2X threat-modeling research. Prior CTF work shows that hands-on challenge environments are effective when they provide progressive difficulty, low-friction access, and sustained engagement. Automotive security

Table 10: Effectiveness metrics for V2X security training.

Dimension	Metric	Result
Engagement	Registered teams that became active	21/31
Demonstrated skill	Active teams with ≥ 1 correct challenge solve	16/21
Challenge-set completion	Active teams that solved all challenges	5/21
Conceptual knowledge [†]	Pre-/post-CTF change in baseline V2X security knowledge	+6%

[†]Preliminary and underpowered: based on $n=6$ post-CTF survey responses.

competitions demonstrate clear interest in domain-specific training, but most focus on traditional in-vehicle components such as CAN, firmware, and infotainment systems, often through hardware-heavy or closed deployments. In parallel, the V2X literature has developed detailed attack taxonomies, risk models, and defense mechanisms, yet these contributions rarely translate into reusable educational environments. Our work is motivated by this gap between what the community knows about CAV/V2X threats and what learners can currently practice in a scalable, reproducible setting. Whereas Section 3.2 evaluated existing platforms against our CAV CTF *requirements*, this section positions *ConCAVe* within the broader *research* landscape of automotive-security training and V2X threat modeling.

6.1 Automotive CTFs, Testbeds and Simulation Platforms

Prior automotive-security CTFs and training environments establish the value of hands-on exercises, but they primarily target conventional vehicle-security surfaces rather than the connected-vehicle ecosystem. Event-based studies have examined infotainment and CAN-centric attacks through Automotive Grade Linux [24, 17] and real-vehicle attack-defense competitions [26]. Hardware and software testbeds such as PASTA [35] and RAMN [46] lower the barrier to experimenting with ECUs, firmware, and in-vehicle networks. Industry competitions and commercial platforms, including Pwn2Own Automotive, DEF CON automotive events, and VicOne/Blockharbor offerings [50], demonstrate sustained interest in automotive-security training. Finally, simulation platforms such as Gazebo [27], Auto-Drive [44], LGSVL [41], and MetaDrive [28] provide reusable cyber-physical environments for vehicle experimentation, although they are not CTF platforms by themselves.

Across these efforts, the common limitation is that automotive-security training remains centered on in-vehicle components, specialized hardware, or standalone simulation environments rather than open, scalable, and rehostable V2X-centered attack scenarios. *ConCAVe* differs by treating V2X security as a first-class training target and by packaging connected-vehicle attack scenarios within a containerized CTF infrastructure.

6.2 V2X Threat Modelling

A substantial amount of research has characterized the attack surface of connected vehicles and proposed various attacks. Garcia *et al.* [15] provide a comprehensive study of bugs in Automated Vehicle software systems and the root causes for these bugs. Yoshizawa *et al.* [56] provide a detailed analysis of the IEEE 1609.2 [21], ETSI ITS-G5 [12] and 3GPP C-V2X [1] standards and the security mechanisms they provide. They also analyze the attack surface of these standards and the attacks that can be performed on them. Barletta *et al.* [3] extend the MARISMA risk analysis framework to connected vehicles and provide a methodology for risk assessment aligned with ISO/SAE 21434 [22] standards.

On the offensive side, Ansari *et al.* [2] introduce VASP, an application spoofing platform that defines 68 attack scenarios against Basic Safety Message (BSM) and CAM applications, providing the most comprehensive attack catalog for V2X message-level threats. Kamel *et al.* [25] present F2MD, an open-source misbehavior detection framework for C-ITS that integrates plausibility checks, machine-learning-based anomaly detection, and pseudonym change policies within the VEINS/SUMO [42] simulation stack for ITS-G5 and C-V2X.

These works demonstrate that V2X security is a rich and active research area with well-characterized threats and evolving defenses but that usually stops at analysis, taxonomy, and detection frameworks. None of the works address the training dimension and comment on reusable education frameworks i.e., how developers learn to identify, exploit, or defend against these threats in hands-on settings. *ConCAVe* bridges this gap by translating documented V2X attack vectors into interactive CTF challenges backed by a realistic simulation environment; this makes it easy for developers to quickly instantiate an attack vector and train on how it can be attacked.

7 Limitations

Although we had a healthy balance of novice and expert players in the CTF, *ConCAVe* consisted of only beginner-level challenges that covered a handful of attack vectors to mainly evaluate a beginner’s approach towards the challenges.

7.1 Misbehavior Detection

The CTF does not have an exhaustive misbehavior detection. We implemented rudimentary state machine systems to check for vehicle state. This helps us get the challenge up and running and not worry about a standardized system to validate world state. In particular, the current implementation can test whether participants can inject plausible message contents, but it cannot test whether an attack would evade physics-based plausibility checks, consistency checks, or cooperative misbehavior detection. Existing frameworks such as F2MD [25] could be integrated in future versions to support detection-evasion challenges as first-class V2X security exercises.

7.2 Protocol and PKI Coverage

Our implementation uses a simplified CAM message format and does not include digital signature verification or PKI infrastructure. Real V2X deployments rely on SCMS, Cooperative ITS Credentials Management System (CCMS) for certificate management, enabling

attack categories we cannot currently test without modifications to the framework. *ConCAVe* models the application-level message exchange needed for training, but it does not provide standards-complete DSRC or C-V2X systems. These choices keep the platform deployable and easy to extend, but they limit our ability to study certain classes of attacks.

7.3 Simulator Fidelity.

CARLA provides realistic vehicle dynamics and sensor modeling, our V2X layer does not enforce real-time latency constraints. Safety-critical V2X applications require extremely fast message processing, and cryptographic operations (ECDSA signing/verification) add computational overhead not currently modeled. The live event also showed that simulator stability remains the main participant-facing reliability bottleneck, even when the surrounding control plane and container infrastructure operate reliably. This limits our ability to evaluate timing-based attacks or validate that security mechanisms meet deployment requirements.

Despite these limitations, we believe that *ConCAVe* is positioned as a *training platform* for V2X security concepts rather than a standardized validation tool. The challenges serve as an accessible entry point into the CAV security domain, demonstrating the feasibility of simulation-based CTFs and gathering community feedback on challenge design. Future iterations can systematically address these gaps, with more protocol implementations and misbehavior detection frameworks as immediate priorities to enable cryptographic attack categories and detection-evasion challenges that are central to V2X security research.

8 Conclusion

In this paper, we presented *ConCAVe*, an open platform for running CAV CTFs. We showed that *ConCAVe* can model several core V2X threat classes, support new challenge development with limited platform changes, and run a live multi-team event using commodity cloud infrastructure.

Overall, *ConCAVe* demonstrates that open, reproducible, and scalable CAV CTFs are feasible without specialized vehicle hardware, and provides a foundation for future research on connected-vehicle security education, challenge design, simulator-backed cyber-physical training, and more realistic V2X attack evaluation.

References

- [1] 3GPP. 2020. Security aspects of 3GPP support for advanced Vehicle-to-Everything (V2X) services. Technical Specification TS 33.536. 3rd Generation Partnership Project (3GPP). <https://www.3gpp.org/dynareport/33536.htm>.
- [2] Mohammad Raashid Ansari, Jonathan Petit, Jean-Philippe Monteuiis, and Cong Chen. 2023. VASP: v2x application spoofing platform. In *Proceedings Inaugural International Symposium on Vehicle Security & Privacy*. Inaugural International Symposium on Vehicle Security & Privacy. Internet Society, San Diego, CA, USA. ISBN: 978-1-891562-88-4. doi:10.14722/vehiclesec.2023.23071.
- [3] Vita Santa Barletta, Danilo Caivano, Mirko De Vincentiis, Luis Enrique Sánchez, Antonio Santos-Olmo, and Manuel A. Serrano. 2024. Marisma-car: a preliminary approach to risk assessment in automotive security. In *2024 IEEE 21st International Conference on Mobile Ad-Hoc and Smart Systems (MASS)*. (Sept. 2024), 574–579. doi:10.1109/MASS62177.2024.00091.
- [4] [n. d.] Car hacking village. [Online; accessed 2026-04-29]. (). <https://www.carhackingvillage.com/>.
- [5] Kevin Chung and Julian Cohen. 2014. Learning obstacles in the capture the flag model. In *2014 USENIX Summit on Gaming, Games, and Gamification in Security Education (3GSE 14)*.

- [6] [n. d.] Ctfdd : the easiest capture the flag platform. <https://ctfdd.io/>. (Accessed on 11/22/2024). ().
- [7] ctfime team. [n. d.] Ctfime.org / all about ctf (capture the flag). [Online; accessed 2026-04-29]. (). <https://ctfime.org/>.
- [8] 2024. Cyberphysical systems (cps) | nsf - u.s. national science foundation. [Online; accessed 2026-04-29]. (June 2024). <https://www.nsf.gov/funding/opportunities/cps-cyberphysical-systems/503286/nsf24-581>.
- [9] [n. d.] Cybertruck challenge – connecting next generation talent with the heavy duty trucking industry to keep vehicles secure. (). <https://www.cybertruckchallenge.org/>.
- [10] Jason A. Donenfeld. 2017. WireGuard: Next Generation Kernel Network Tunnel. In *Network and Distributed System Symposium (NDSS)*.
- [11] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. 2017. CARLA: An open urban driving simulator. In *Proceedings of the 1st Annual Conference on Robot Learning*, 1–16.
- [12] ETSI. 2019. Intelligent Transport Systems (ITS); ITS-G5 Access layer specification for Intelligent Transport Systems operating in the 5 GHz frequency band. European Standard EN 302 663. European Telecommunications Standards Institute. https://www.etsi.org/deliver/etsi_en/302600_302699/302663/01_03_01_60/en_302663v010301p.pdf.
- [13] ETSI. 2019. Intelligent Transport Systems (ITS); Vehicular Communications; Basic Set of Applications; Part 2: Specification of Cooperative Awareness Basic Service. European Standard EN 302 637-2. European Telecommunications Standards Institute. https://www.etsi.org/deliver/etsi_en/302600_302699/30263702/01_04_01_60/en_30263702v010401p.pdf.
- [14] [n. d.] Exploiting the tesla wall connector from its charge port connector. Retrieved Apr. 23, 2026 from <https://www.synacktiv.com/en/publications/exploiting-the-tesla-wall-connector-from-its-charge-port-connector#>.
- [15] Joshua Garcia, Yang Feng, Junjie Shen, Sumaya Almanee, Yuan Xia, and [and] Qi Alfred Chen. 2020. A comprehensive study of autonomous vehicle bugs. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE '20)*. Association for Computing Machinery, New York, NY, USA, (Oct. 1, 2020), 385–396. ISBN: 978-1-4503-7121-6. doi:10.1145/3377811.3380397.
- [16] 2025. Hackers block us road with 50 self-driving cars in cyber attack - gadget review. [Online; accessed 2026-04-29]. (Oct. 2025). https://www.gadgetreview.com/hackers-block-us-road-with-50-self-driving-cars-in-cyber-attack?utm_source=chatgpt.com.
- [17] [n. d.] Home - automotive grade linux. <https://www.automotivelinux.org/>. ().
- [18] [n. d.] Home | zero day initiative. [Online; accessed 2025-02-13]. (). <https://www.zerodayinitiative.com/>.
- [19] [n. d.] How connected vehicles work | us department of transportation. [Online; accessed 2026-04-30]. (). https://www.transportation.gov/research-and-technology/how-connected-vehicles-work?utm_source=chatgpt.com.
- [20] [n. d.] Ictf: the international capture the flag competition | ictf-website. [Online; accessed 2026-04-29]. (). <https://ictf.cs.ucsb.edu/>.
- [21] [n. d.] IEEE standards association. IEEE Standards Association. Retrieved Mar. 24, 2026 from <https://standards.ieee.org/ieee/1609/2/10258/>.
- [22] [n. d.] Iso/sae21434.d1 : road vehicles - cybersecurity engineering - sae international. [Online; accessed 2026-04-29]. (). <https://www.sae.org/standards/s/iso21434d1-road-vehicles-cybersecurity-engineering>.
- [23] [n. d.] J3016_202104 : taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles - sae international. [Online; accessed 2026-04-30]. (). https://www.sae.org/standards/j3016_202104-taxonomy-definitions-terms-related-driving-automation-systems-road-motor-vehicles.
- [24] Seonghoon Jeong, Minsoo Ryu, Hyunjae Kang, and Huy Kang Kim. 2023. Infotainment system matters: understanding the impact and implications of in-vehicle infotainment system hacking with automotive grade linux. en. In *Proceedings of the Thirteenth ACM Conference on Data and Application Security and Privacy*. ACM, Charlotte NC USA, (Apr. 2023), 201–212. ISBN: 9798400700675. doi:10.1145/3577923.3583650.
- [25] J. Kamel, M. R. Ansari, J. Petit, A. Kaiser, I. Ben Jemaa, and P. Urien. 2020. Simulation framework for misbehavior detection in vehicular networks. *IEEE Transactions on Vehicular Technology*.
- [26] Hyunjae Kang, Byung Il Kwak, Young Hun Lee, Haneol Lee, Hwejae Lee, and Huy Kang Kim. 2021. Car Hacking and Defense Competition on In-Vehicle Network. In *Proceedings Third International Workshop on Automotive and Autonomous Vehicle Security*. Third International Workshop on Automotive and Autonomous Vehicle Security. Internet Society, Virtual. ISBN: 978-1-891562-68-6. doi:10.14722/autosec.2021.23035.
- [27] Nathan Koenig and Andrew Howard. 2004. Design and use paradigms for gazebo, an open-source multi-robot simulator. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*. Sendai, Japan, (Sept. 2004), 2149–2154.
- [28] Quanyi Li, Zhenghao Peng, Lan Feng, Qihang Zhang, Zhenghai Xue, and Bolei Zhou. 2022. Metadrive: composing diverse driving scenarios for generalizable reinforcement learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- [29] Emmanouil Magkos. 2020. An analysis and evaluation of open source capture the flag platforms as cybersecurity e-learning tools. *Information security education. information security in action: 13th Ifip Wg*, 579.
- [30] Lucas McDaniel, Erik Talvi, and Brian Hay. 2016. Capture the flag as cyber security introduction. In *2016 49th hawaii international conference on system sciences (hicss)*. IEEE, 5479–5486.
- [31] [n. d.] Mobisec - mobile security course. <https://mobisec.reyammer.io/>. ().
- [32] Connor Nelson and Yan Shoshitaishvili. 2024. DOJO: Applied Cybersecurity Education in the Browser. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1. SIGCSE 2024: The 55th ACM Technical Symposium on Computer Science Education*. ACM, Portland OR USA, (Mar. 7, 2024), 930–936. ISBN: 979-8-4007-0423-9. doi:10.1145/3626252.3630836.
- [33] Connor Nelson and Yan Shoshitaishvili. 2024. PWN The Learning Curve: Education-First CTF Challenges. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1. SIGCSE 2024: The 55th ACM Technical Symposium on Computer Science Education*. ACM, Portland OR USA, (Mar. 7, 2024), 937–943. ISBN: 979-8-4007-0423-9. doi:10.1145/3626252.3630912.
- [34] Sen Nie, Ling Liu, and Yuefeng Du. [n. d.] Free-fall-hacking-tesla-from-wireless-to-can-bus.
- [35] [n. d.] Pasta-auto/pasta1.0: pasta: portable automotive security testbed with adaptability. <https://github.com/pasta-auto/PASTA1.0>. (Accessed on 11/15/2024). ().
- [36] [n. d.] Picocf - cmu cybersecurity competition. [Online; accessed 2026-04-30]. (). <https://picocf.org/>.
- [37] [n. d.] Pwn.college. <https://pwn.college/>. ().
- [38] [n. d.] Radareorg/r2wars: corewars but within r2. [Online; accessed 2026-04-29]. (). <https://github.com/radareorg/r2wars>.
- [39] Raghu Raman, Sherin Sunny, Vipin Pavithran, and Krishnasree Achuthan. 2014. Framework for evaluating capture the flag (ctf) security competitions. In *International Conference for Convergence for Technology-2014*. IEEE, 1–5.
- [40] Riscure. 2017. RHme3: Riscure Hack Me 3 – Automotive Edition. <https://ctftime.org/event/502/>. [Online; accessed 22-July-2026]. (2017).
- [41] Guodong Rong et al. 2020. Lgsvl simulator: a high fidelity simulator for autonomous driving. *arXiv preprint arXiv:2005.03778*.
- [42] Christoph Sommer, Reinhard German, and Falko Dressler. 2011. Bidirectionally coupled network and road traffic simulation for improved IVC analysis. *IEEE Transactions on Mobile Computing*, 10, 1, (Jan. 2011), 3–15. doi:10.1109/TMC.2010.133.
- [43] The Dark Tangent. [n. d.] Def con@ hacking conference - capture the flag archive. [Online; accessed 2026-04-29]. (). <https://defcon.org/html/links/dc-ctf.html>.
- [44] [n. d.] Tinker-twins/autodrive: an integrated cyber-physical ecosystem for autonomous driving research and education. <https://github.com/Tinker-Twins/AutoDRIVE>. (Accessed on 11/19/2024). ().
- [45] Contributors to Wikimedia projects. 2021. Capture the flag (cybersecurity) - wikipedia. [Online; accessed 2026-04-29]. (Nov. 2021). [https://en.wikipedia.org/wiki/Capture_the_flag_\(cybersecurity\)](https://en.wikipedia.org/wiki/Capture_the_flag_(cybersecurity)).
- [46] [n. d.] Toyotainfotech/ramn: ramn (resistant automotive miniature network), a miniature can/can-fd testbed of 4 electronic control units. <https://github.com/ToyotaInfoTech/RAMN>. (Accessed on 11/15/2024). ().
- [47] US Department of Transportation. 2026. Interactive connected vehicle deployment map. <https://dot.ca.gov/programs/traffic-operations/cav>. [Online; accessed 29-April-2026]. (2026).
- [48] [n. d.] Us dot pilot deployment program 2024 update. [Online]. (). <https://www.itskrs.its.dot.gov/briefings/executive-briefing/connected-vehicle-pilot-deployment-program-2024-update/>.
- [49] [n. d.] Usdot awards nearly \$60 million in advanced vehicle technology grants to arizona, texas and utah to serve as national models and help save lives on our nation's roadways | fhwa. [Online; accessed 2026-04-29]. (). <https://highways.dot.gov/newsroom/usdot-awards-nearly-60-million-advanced-vehicle-technology-grants-arizona-texas-and-utah>.
- [50] [n. d.] Vicone – automotive ctf - vicone. <https://vicone.com/automotive-ctf>. (Accessed on 11/22/2024). ().
- [51] VicOne and Block Harbor. 2024. VicOne and Block Harbor Spearhead Biggest Automotive Capture the Flag Competition for Cybersecurity Enthusiasts Worldwide. <https://www.businesswire.com/news/home/20240829807452/en/>. [Online; accessed 22-July-2026]. (2024).
- [52] Giovanni Vigna, Kevin Borgolte, Jacopo Corbetta, Adam Doupe, Yanick Fratantonio, Luca Invernizzi, Dhilung Kirat, and Yan Shoshitaishvili. [n. d.] Ten Years of iCTF: The Good, The Bad, and The Ugly.
- [53] [n. d.] Vsec, blockharbor. (). <https://www.blockharbor.io/vsec-platform/>.

- [54] Wikipedia contributors. 2026. Cellular v2x — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Cellular_V2X&oldid=1340636217. [Online; accessed 13-March-2026]. (2026).
- [55] Wikipedia contributors. 2025. Dedicated short-range communications — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Dedicated_short-range_communications&oldid=1305299462. [Online; accessed 13-March-2026]. (2025).
- [56] Takahito Yoshizawa, Dave Singelee, Jan Tobias Muehlberg, Stephane Delbruel, Amir Taherkordi, Danny Hughes, and Bart Preneel. 2023. A survey of security and privacy issues in v2x communication systems. *ACM Comput. Surv.*, 55, 9, (Jan. 16, 2023), 185:1–185:36. doi:10.1145/3558052.
- [57] Zero Day Initiative. 2026. Pwn2Own Automotive 2026 – Day Three Results and the Master of Pwn. <https://www.thezdi.com/blog/2026/1/23/pwn2own-automotive-2026-day-three-results-and-the-master-of-pwn>. [Online; accessed 22-July-2026]. (2026).

9 Open Science

The artifacts needed to evaluate the core contributions of this submission are available at the following URL: <https://anonymous.4open.science/r/ConCAVe-EA9D>